

# Otimização de balanceadores de carga de sistemas de distribuição de conteúdo por meio de monitoramento sistemático de memória

Pedro Faria Fernandes

Universidade do Estado de Santa Catarina

28 de novembro de 2025

# Sumário I

Introdução

Fundamentação Teórica

Trabalhos Relacionados

Proposta

Implementação

Experimentos e Resultados

Conclusão

Referências

# Introdução

# Contexto e Motivação

## Evolução do Consumo de Vídeo

- ▶ Da **televisão** tradicional para a Internet;
- ▶ Consolidação do **vídeo sob demanda** como padrão;
- ▶ Crescimento **exponencial** do tráfego de vídeo na rede;

## Distribuição de Conteúdo

- ▶ Replicação de conteúdo em múltiplos servidores;
- ▶ Redes de Distribuição de Conteúdo (**CDNs**) como solução central;
- ▶ Necessidade de **otimizar recursos** para manter a Qualidade de Experiência (QoE);

# O Problema

- ▶ Sistemas de distribuição de conteúdo  $\approx$  múltiplos servidores e um **balanceador de carga** central;
- ▶ Dispositivos de armazenamento persistente (discos) frequentemente são **gargalos** em cenários de *streaming*;
- ▶ Uso intensivo de **memória principal** como *cache* alivia o disco, mas torna a memória um recurso crítico;
- ▶ Muitas estratégias clássicas de balanceamento de carga ignoram o estado real de memória dos servidores;
- ▶ **Lacuna**: ausência de algoritmos que utilizem **exclusivamente** métricas de memória (RAM e I/O de disco) como parâmetro principal de decisão.

# Objetivo

## Objetivo do trabalho

- ▶ Avaliar uma estratégia de otimização da escolha de servidores em balanceadores de carga para sistemas de distribuição de conteúdo, baseada **exclusivamente** no uso de memória;
- ▶ Implementar um algoritmo probabilístico de balanceamento de carga, apoiado em uma arquitetura de simulação completa;
- ▶ Realizar análise comparativa com algoritmos clássicos (*Round-Robin* e Seleção Aleatória) sob diferentes cenários de carga e heterogeneidade.

## Fundamentação Teórica

# Fundamentação Teórica

## Tópicos Centrais

- ▶ Balanceadores de carga em diferentes camadas (rede e aplicação) e balanceamento dinâmico;
- ▶ Padrão **MPEG-DASH** para *streaming* adaptativo sobre HTTP;
- ▶ Arquitetura de **CDNs** e servidores de distribuição de conteúdo;
- ▶ Monitoramento de memória via APIs de sistemas operacionais;
- ▶ Interpretação de dados de carga desatualizados;
- ▶ Virtualização de redes baseada em *containers* para simulação controlada.



# Balanceadores de Carga

Balanceadores de carga são cruciais em sistemas distribuídos, incluindo CDNs.

## Função e Importância

- ▶ Distribuem requisições entre múltiplos servidores, habilitando **escalabilidade horizontal** [10];
- ▶ Aumentam desempenho, disponibilidade e tolerância a falhas;
- ▶ Podem atuar no **nível de rede** (L3/L4) ou no **nível de aplicação** (L7) [18];
- ▶ Algoritmos clássicos: *Round-Robin*, *Seleção Aleatória*, *Least Connections*, entre outros.

# MPEG-DASH

**HTTP Streaming** é o padrão para distribuição de multimídia, culminando no **DASH** (*Dynamic Adaptive Streaming over HTTP*) [4].

## Componentes Principais

- ▶ **MPD (Media Presentation Description)**: manifesto XML com metadados de representações e URLs [20];
- ▶ Conteúdo segmentado em **blocos** requisitados via HTTP GET, altamente *cacheáveis* [21];
- ▶ Cliente decide **quando** e **em que qualidade** baixar cada segmento;
- ▶ Integra-se naturalmente com CDNs e balanceadores de carga orientados a HTTP.

# CDNs e Servidores de Conteúdo

## Content Delivery Networks (CDNs)

- ▶ Sistemas distribuídos que replicam conteúdo em servidores de borda, próximos ao usuário [19];
- ▶ Arquitetura com servidores de origem e **servidores de borda** em múltiplos PoPs;
- ▶ Uso intensivo de **cache** para reduzir latência e carga na infraestrutura central;
- ▶ Dependem fortemente de **balanceamento de carga** eficiente.



# Monitoramento de Memória

## APIs de Sistemas Operacionais

- ▶ Kernels mantêm estatísticas detalhadas de memória principal e I/O de disco;
- ▶ **Linux**: arquivos como `/proc/meminfo` e `/proc/diskstats` e interfaces de *cgroups* [23, 22];
- ▶ **Windows** e **macOS** expõem contadores e APIs específicas [14, 15, 2, 3];
- ▶ Fornecem base para medir **uso de RAM** e **taxa de leitura de disco** por servidor ou contêiner.

# Interpretação de Dados Desatualizados

- ▶ Em sistemas distribuídos, informações de carga chegam ao balanceador com **atraso**;
- ▶ Dados desatualizados podem **sobrecarregar** servidores em vez de evitá-lo [8, 16];
- ▶ Alternativas ingênuas: ignorar dados de carga (algoritmos cegos) ou usar apenas um subconjunto aleatório de servidores;
- ▶ O trabalho *Interpreting Stale Load Information* propõe algoritmos que **interpretam** a carga em função de sua idade, como o **Basic LI** [7];
- ▶ Este conceito é adaptado neste TCC para métricas de memória.

# Virtualização de Redes e Containers

- ▶ **Virtualização de redes:** múltiplas redes virtuais sobre a mesma infraestrutura física [6];
- ▶ **Containers Linux:** permitem múltiplos ambientes isolados compartilhando o mesmo kernel;
- ▶ Ferramentas como **Docker** e LXC reduzem *overhead* em comparação com VMs completas;
- ▶ Viabilizam criação de **ambientes de teste reproduzíveis**, com controle fino sobre recursos (cgroups).

## Trabalhos Relacionados



# Trabalhos Relacionados (Parte 1)

| Título                          | Proposta                                                                                                       | Limitações                                                                                                              |
|---------------------------------|----------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| NGINX [12, 17]                  | Algoritmos clássicos de balanceamento HTTP ( <i>Round-Robin</i> , <i>Least Connections</i> , <i>IP hash</i> ). | Não usa métricas de <b>memória</b> e I/O de disco para decisões de roteamento.                                          |
| HAProxy [9]                     | Balanceador TCP/HTTP de alta eficiência e baixo consumo de recursos.                                           | Não oferece balanceamento nativo guiado por <b>uso de memória</b> dos servidores.                                       |
| Docker Swarm (memória) [5]      | Acrescenta consumo de <b>memória</b> como fator no balanceamento interno do Swarm.                             | Restrito ao Docker; não trata CDNs nem métricas diretas de QoE em vídeo.                                                |
| Algoritmo dinâmico em cloud [1] | Usa múltiplas métricas (CPU, memória, largura de banda, etc.) para lidar com <i>flash crowds</i> .             | Focado em aplicações <b>cloud</b> genéricas; não é específico para servidores de conteúdo nem para gargalos de memória. |

# Trabalhos Relacionados (Parte 2)

| Título                               | Proposta                                                                                    | Limitações                                                                                          |
|--------------------------------------|---------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| Revisão de algoritmos dinâmicos [13] | Compara 15 algoritmos de balanceamento dinâmico para servidores web.                        | Poucos usam memória/disco, e nenhum trata esses recursos como <b>parâmetro principal</b> exclusivo. |
| iSCON para vídeo [11]                | Replica vídeos populares e balanceia carga com base na <b>largura de banda</b> de saída.    | Considera predominantemente banda de rede; ignora gargalos de <b>memória</b> e disco.               |
| Servidores de vídeo distrib. [24]    | Duas fases: alocação inicial de réplicas e <i>load shifting</i> por <b>streams ativos</b> . | Mede carga apenas por número de streams; ignora uso real de <b>memória</b> e I/O de disco.          |

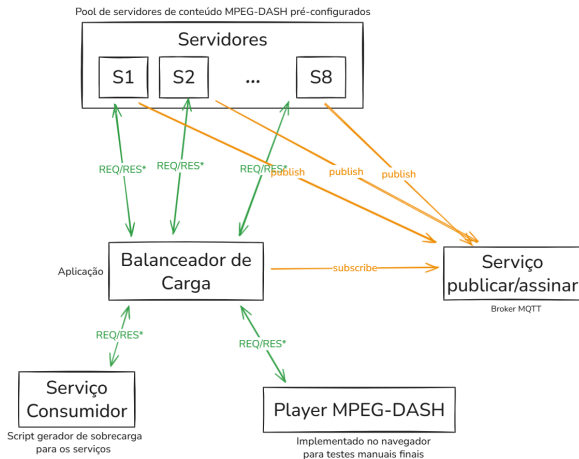
# Proposta

# Requisitos Tecnológicos

## Componentes Principais

- ▶ **Servidores de Conteúdo:** 8 servidores MPEG-DASH, com capacidades heterogêneas de memória e disco;
- ▶ **Balanceador de Carga:** Implementado em *Rust*, responsável por aplicar os algoritmos de seleção de servidor;
- ▶ **Cliente Gerador de Carga:** Aplicação em *Python* que simula múltiplos usuários MPEG-DASH concorrentes;
- ▶ **Servidor MQTT:** *Broker* NanoMQ utilizado para transporte assíncrono de métricas entre servidores e balanceador.

# Arquitetura Geral do Sistema



\* Requisição e Resposta de conteúdo em áudio/vídeo

Arquitetura geral do sistema experimental (autor, 2025).

# Objetivo do Algoritmo

- ▶ Definir o método de escolha de um servidor para cada nova requisição de vídeo;
- ▶ Algoritmo de natureza **probabilística**, baseado no **Basic Load Interpretation** [7];
- ▶ Probabilidade de escolha de cada servidor ajustada dinamicamente de acordo com:
  - ▶ Consumo de **memória RAM**;
  - ▶ Taxa de entrada e saída de **disco** (I/O);
  - ▶ **Idade** das informações de carga.
- ▶ Servidores publicam periodicamente suas métricas de carga para o balanceador.

# Cálculo da Probabilidade

A probabilidade ( $P_i$ ) de uma requisição ser enviada para um servidor  $i$  é baseada no **Basic Load Interpretation**:

$$P_i = \frac{\frac{L_{tot} + arrive_T}{n} - L_i}{arrive_T} \quad (1)$$

$P_i$  : Probabilidade de escolha do servidor  $i$ ;

$n$  : Número total de servidores disponíveis;

$L_i$  : Carga individual reportada pelo servidor  $i$ ;

$L_{tot}$  : Somatório de todas as cargas reportadas ( $\sum L_i$ );

$arrive_T$  : Número esperado de novas requisições durante o intervalo médio  $T$  das informações de carga.

## Cálculo de $arrive_T$

O parâmetro  $arrive_T$  representa o número esperado de requisições durante o intervalo de desatualização:

$$arrive_T = \left( \frac{R_{total}}{T_{elapsed}} \right) \cdot T_{stale} \cdot \left( \frac{L_{tot}}{R_{active}} \right) \cdot C \quad (2)$$

$R_{total}$  : Total de requisições recebidas pelo balanceador;

$T_{elapsed}$  : Tempo total de execução do balanceador;

$T_{stale}$  : Tempo médio de desatualização das métricas;

$R_{active}$  : Requisições ativas ( $\sum_{i=1}^n AR_i$ );

$C$  : Constante de ajuste (definida heurísticamente).



# Adaptação do Parâmetro de Carga ( $L_i$ )

No contexto deste trabalho, a **carga** ( $L_i$ ) é medida a partir do uso de memória:

$$L_i = CM \cdot LM_i + CD \cdot LD_i \quad (3)$$

$LM_i$  : Consumo de **memória** normalizado do servidor  $i$  (0–1);

$LD_i$  : Carga de **disco** (I/O) normalizada do servidor  $i$  (0–1);

$CM, CD$  : Coeficientes dinâmicos que refletem a importância relativa de memória e disco.

# Cálculo dos Coeficientes Dinâmicos ( $CM$ e $CD$ )

Os coeficientes aumentam o peso do recurso mais pressionado no sistema como um todo.

- Taxa média de uso de memória:

$$TM = \frac{1}{n} \sum_{i=1}^n LM_i$$

- Taxa média de uso de disco:

$$TD = \frac{1}{n} \sum_{i=1}^n LD_i$$

$$CM = \frac{TM}{TM + TD} \quad (4)$$

$$CD = \frac{TD}{TM + TD} \quad (5)$$

# Pseudocódigo — Atualização da Distribuição

---

**Algorithm** Atualização da distribuição de probabilidade

---

**Gatilho:** Nova informação de carga via MQTT.

**Require:** Métricas  $LM_i, LD_i, T_i, AR_i$  para cada servidor  $S_i$

**Ensure:** Probabilidades  $P_i$  atualizadas

- 1: **procedure** ATUALIZARDISTRIBUICAO( $LM_i, LD_i, T_i, AR_i$ )
  - 2:   Atualizar métricas do servidor  $S_i$
  - 3:    $TM \leftarrow \frac{1}{n} \sum_{i=1}^n LM_i, \quad TD \leftarrow \frac{1}{n} \sum_{i=1}^n LD_i$
  - 4:    $T \leftarrow \frac{1}{n} \sum_{i=1}^n (T_{atual} - T_i)$
  - 5:   Calcular  $CM$  e  $CD$  (Eq. 4, 5)
  - 6:   Calcular  $L_i$  para cada servidor (Eq. 3)
  - 7:    $L_{tot} \leftarrow \sum_{i=1}^n L_i$
  - 8:   Calcular  $arrive_T$  (Eq. 2)
  - 9:   Calcular  $P_i$  para cada servidor (Eq. 1)
  - 10:   **if**  $n = 0$  **ou**  $arrive_T \leq 0$  **then**
  - 11:      $P_i \leftarrow 1/n$  para todos os servidores
-

# Implementação

# Ambiente Experimental

- ▶ Ambiente executado em uma única máquina física (**Windows** + **WSL2** com kernel Linux completo);
- ▶ Todos os componentes são executados em contêineres **Docker**;
- ▶ Uso de **cgroups v2** para monitorar e limitar recursos (memória, I/O de disco);
- ▶ Limpeza do *page cache* antes de cada bateria de testes para garantir reprodutibilidade.

# Organização do Conteúdo e Bind Mounts

- ▶ Conteúdo de vídeo armazenado em `/data` no *host* WSL, com diretórios específicos para cada servidor:
  - ▶ `/data/server-1`, `/data/server-2`, ..., `/data/server-8`;
- ▶ Cada servidor MPEG-DASH monta seu diretório via **bind mount** Docker;
- ▶ Simula cenário real de CDNs com **conteúdo exclusivo** por servidor de borda;
- ▶ Evita cópia de arquivos para dentro das imagens, reduzindo tamanho e facilitando atualização.

# Módulos Principais e Auxiliares

## Módulos Principais

- ▶ **Servidores MPEG-DASH** (*ASP.NET Core*): servem segmentos de vídeo e publicam métricas de recursos;
- ▶ **Balanceador de Carga** (*Rust*): *proxy* reverso HTTP com estratégias *Round-Robin*, Aleatória e Monitoramento de Memória;
- ▶ **Gerador de Carga** (*Python/aiohttp*): simula múltiplos clientes concorrentes.

## Módulos Auxiliares

- ▶ **Broker MQTT** (*NanoMQ*);
- ▶ **Coletor de Métricas** (API em Python);
- ▶ **Painel de Monitoramento** em tempo real (HTML + *Chart.js*);
- ▶ **Cliente de Validação** MPEG-DASH (*dash.js*).

# Monitoramento e Publicação de Métricas

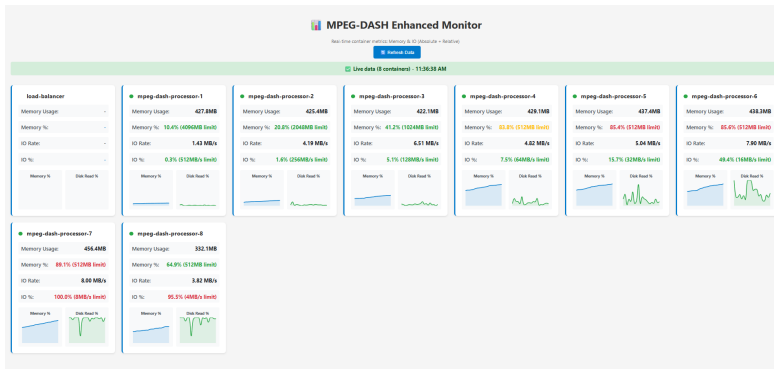
- ▶ Servidores MPEG-DASH executam um serviço de fundo MetricsPublisher;
- ▶ Leituras periódicas de:
  - ▶ `/sys/fs/cgroup/memory.current` (memória utilizada);
  - ▶ `/sys/fs/cgroup/io.stat` (bytes lidos de disco);
- ▶ Métricas são normalizadas em relação aos limites configurados para cada contêiner;
- ▶ Publicação em tópicos MQTT específicos, consumidos pelo balanceador de carga;
- ▶ Intervalo de publicação configurável (por exemplo, 6s e 30s) para avaliar impacto de dados desatualizados.



# Geração de Conteúdo MPEG-DASH

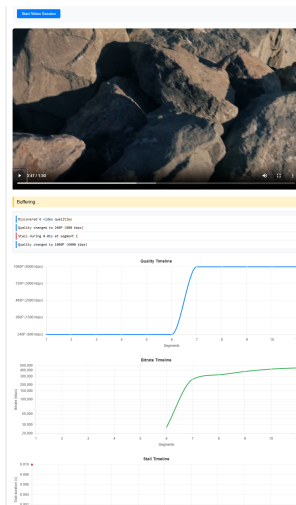
- ▶ Conteúdo gerado com `ffmpeg`, a partir de um vídeo de referência;
- ▶ Criação de múltiplas representações (1080p, 720p, 480p, 360p, 240p, 144p);
- ▶ Segmentação em blocos MPEG-DASH e geração de manifestos MPD;
- ▶ Conteúdo é replicado para os diretórios de cada servidor, preservando estrutura;
- ▶ Permite avaliar impacto do balanceamento na **qualidade adaptativa** do vídeo.

# Painel de Monitoramento em Tempo Real



Painel de monitoramento em tempo real com métricas de memória, I/O de disco e requisições ativas.

# Cliente MPEG-DASH para Validação



Cliente MPEG-DASH (dash.js) com gráficos em tempo real de qualidade, *bitrate* e travamentos (*stalls*).

## Experimentos e Resultados

# Cenários de Teste

| Cenário | Usuários | Distribuição | Configuração dos Servidores                                                                                                                                                                                                 |
|---------|----------|--------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1       | 100      | Heterogênea  | <b>S1:</b> 4096MB / 1024MB/s; <b>S2:</b> 2048MB / 512MB/s; <b>S3:</b> 1024MB / 256MB/s; <b>S4:</b> 512MB / 128MB/s; <b>S5:</b> 256MB / 64MB/s; <b>S6:</b> 128MB / 32MB/s; <b>S7:</b> 64MB / 16MB/s; <b>S8:</b> 32MB / 8MB/s |
| 2       | 100      | Homogênea    | Todos os servidores: 1024MB RAM e 256MB/s de leitura                                                                                                                                                                        |
| 3       | 1000     | Heterogênea  | Mesma configuração do Cenário 1                                                                                                                                                                                             |
| 4       | 1000     | Homogênea    | Mesma configuração do Cenário 2                                                                                                                                                                                             |

# Estratégias de Balanceamento

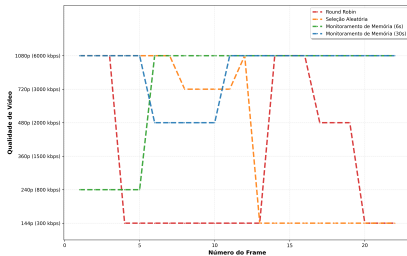
- ▶ **Round-Robin**: distribuição cíclica uniforme;
- ▶ **Seleção Aleatória**: servidor escolhido de forma uniforme e aleatória a cada requisição;
- ▶ **Monitoramento de Memória (6s)**: algoritmo proposto com atualização frequente de métricas;
- ▶ **Monitoramento de Memória (30s)**: algoritmo proposto com dados moderadamente desatualizados.

# Métricas de Avaliação de QoE

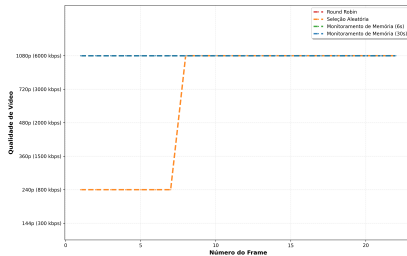
- ▶ **Latência Média** de resposta por requisição HTTP;
- ▶ **Qualidade** de vídeo ao longo da sessão (resoluções escolhidas pelo cliente);
- ▶ **Bitrate** médio por segmento;
- ▶ **Travamentos** (*stalls*): quantidade, duração total e maior duração.

# Qualidade de Vídeo por Cenário

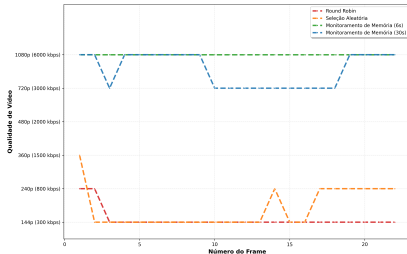
Qualidade de Vídeo por Frame - Cenário 1



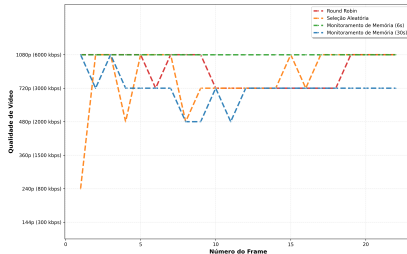
Qualidade de Vídeo por Frame - Cenário 2



Qualidade de Vídeo por Frame - Cenário 3



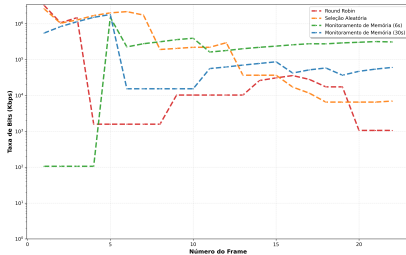
Qualidade de Vídeo por Frame - Cenário 4



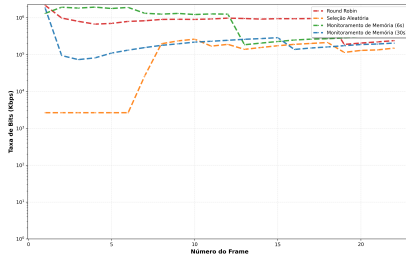


# Bitrate por Cenário

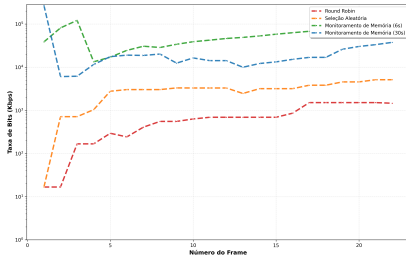
Taxa de Bits por Frame - Cenário 1



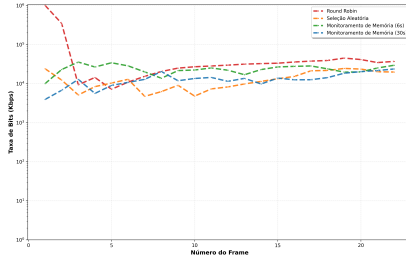
Taxa de Bits por Frame - Cenário 2



Taxa de Bits por Frame - Cenário 3

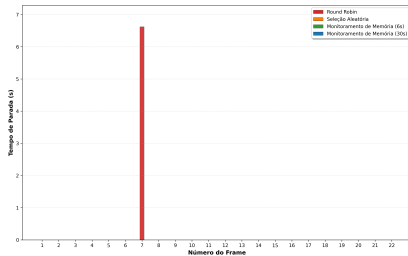


Taxa de Bits por Frame - Cenário 4

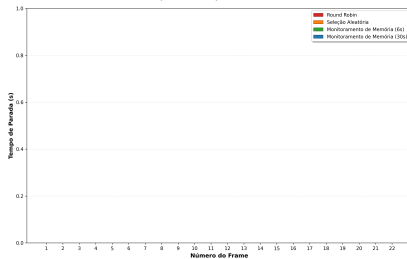


# Stalls por Cenário

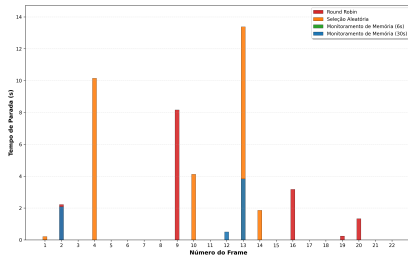
Duração de Parada por Frame - Cenário 1



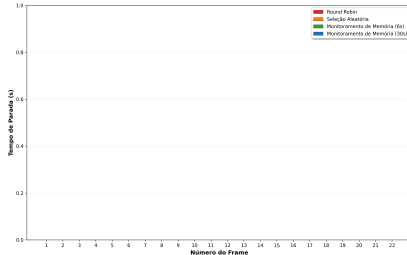
Duração de Parada por Frame - Cenário 2



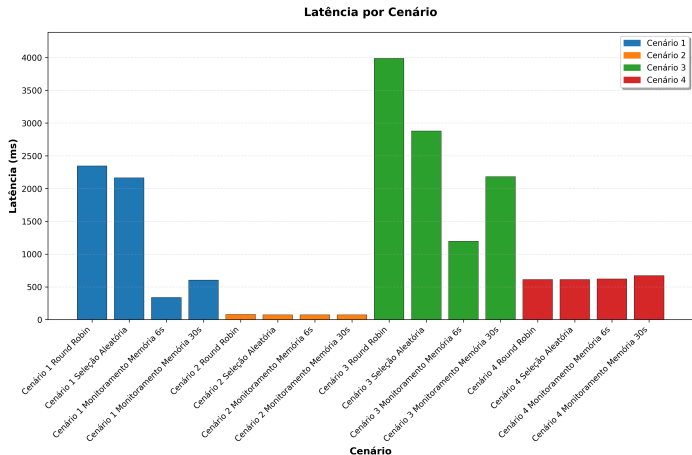
Duração de Parada por Frame - Cenário 3



Duração de Parada por Frame - Cenário 4



# Latência Média por Cenário



Latência média de resposta por cenário e estratégia de balanceamento.

# Síntese dos Resultados

- ▶ Em cenários **heterogêneos** (1 e 3), o algoritmo proposto (**MM 6s**) reduziu a latência média em até **85,7%** em relação ao *Round-Robin*;
- ▶ No Cenário 3, estratégias clássicas apresentaram múltiplos travamentos e queda de qualidade para 144p, enquanto o algoritmo proposto eliminou travamentos e manteve qualidade alta (720p–1080p);
- ▶ A variante com **30s** de atualização, mesmo com dados desatualizados, superou consistentemente as estratégias clássicas;
- ▶ Em cenários **homogêneos** (2 e 4), todas as estratégias apresentaram desempenho similar, evidenciando que o benefício do algoritmo está ligado à heterogeneidade.

## Conclusão

# Validação das Hipóteses

- ▶ **Hipótese 1:** Algoritmo baseado em monitoramento de memória supera *Round-Robin* e Aleatório em ambientes heterogêneos:
  - ▶ Confirmada por reduções expressivas de latência e eliminação de travamentos.
- ▶ **Hipótese 2:** Intervalo de 6s fornece melhores resultados que 30s:
  - ▶ Confirmada, embora 30s ainda apresente desempenho superior às estratégias clássicas.
- ▶ **Hipótese 3:** Robustez com dados moderadamente desatualizados:
  - ▶ Confirmada: o algoritmo mantém bom desempenho mesmo com menor frequência de atualização.

# Fatores Determinantes do Desempenho

- ▶ **Heterogeneidade dos servidores:**

- ▶ Quanto maior a diferença de capacidade entre nós, maior o ganho do balanceamento inteligente;

- ▶ **Nível de concorrência:**

- ▶ Alta concorrência funciona como teste de estresse, expondo limitações de algoritmos simples;
- ▶ O algoritmo proposto mantém desempenho superior mesmo em cenários extremos (1000 usuários).

# Contribuições do Trabalho

- ▶ Proposição de um **algoritmo de balanceamento** baseado em monitoramento de memória e I/O de disco;
- ▶ Adaptação do conceito de **Load Interpretation** para métricas de recursos de baixo nível;
- ▶ Implementação completa e aberta com *C#*, *Rust*, *Python*, Docker e MQTT;
- ▶ Infraestrutura experimental reutilizável para avaliação de algoritmos de balanceamento em *streaming* MPEG-DASH;
- ▶ Demonstração de que heterogeneidade é fator crítico para o valor de estratégias inteligentes de balanceamento.



# Limitações e Trabalhos Futuros

- ▶ Experimentos conduzidos em ambiente virtualizado único, sem latência geográfica real;
- ▶ Uso de um único algoritmo ABR (`dash.js`) e conjunto limitado de cenários de carga;
- ▶ Métricas focadas em memória e disco, sem considerar rede e CPU como co-fatores;
- ▶ **Trabalhos futuros:**
  - ▶ Validação em ambientes distribuídos reais e CDNs de produção;
  - ▶ Extensão para múltiplas métricas (CPU, rede, latência geográfica);
  - ▶ Integração com SDN e *Content Steering* entre múltiplas CDNs;
  - ▶ Aplicação em cenários de *edge computing* e 5G.

# Reprodutibilidade e Considerações Finais

- ▶ Todo o código, scripts e dados estão disponíveis em:
  - ▶ <https://github.com/peedrofernandes/memory-oriented-load-balancer.git>
- ▶ A infraestrutura experimental permite reproduzir todos os 16 cenários de teste;
- ▶ Resultados demonstram que **monitorar e interpretar adequadamente o uso de recursos** é essencial para QoE em CDNs heterogêneas;
- ▶ Estratégias de balanceamento conscientes de memória representam um caminho promissor para a próxima geração de sistemas de distribuição de conteúdo.

## Referências

# Referências I

- [1] Adekunbi Adewojo and Julian Bass. A novel weight-assignment load balancing algorithm for cloud applications. *SN Computer Science*, 4, 03 2023.
- [2] Apple Inc. About the virtual memory system, September 2012. Documentação de arquivo que descreve os conceitos fundamentais do sistema de memória virtual do macOS.
- [3] Apple Inc. Iokit fundamentals: Introduction, September 2013. Documentação de arquivo que apresenta o framework I/O Kit, fundamental para o monitoramento de dispositivos, incluindo I/O de disco.
- [4] Ali Begen, Tankut Akgul, and Mark Baugher. Watching video over the web: Part 1: Streaming protocols. *IEEE Internet Computing*, 15(2), 2011.
- [5] Mochamad Rexa Mei Bella, Mahendra Data, and Widhi Yahya. Web server load balancing based on memory utilization using docker swarm. In *2018 International Conference on Sustainable Information Engineering and Technology (SIET)*, 2018.

# Referências II

- [6] N.M. Mosharaf Kabir Chowdhury and Raouf Boutaba. A survey of network virtualization. *Computer Networks*, 54(5), 2010.
- [7] M. Dahlin. Interpreting stale load information. *IEEE Transactions on Parallel and Distributed Systems*, 11(10), 2000.
- [8] Derek L. Eager, Edward D. Lazowska, and John Zahorjan. Adaptive load sharing in homogeneous distributed systems. *IEEE Transactions on Software Engineering*, SE-12(5), 1986.
- [9] HAProxy. Haproxy - the reliable, high performance tcp/http load balancer, 2025. Acessado em: 27 de março de 2025.
- [10] Kimberly Jane. Scalability issues in database systems: Strategies for scaling databases horizontally and vertically. 03 2025.
- [11] Junyeop Kim and Youjip Won. Dynamic load balancing for efficient video streaming service. In *2015 International Conference on Information Networking (ICOIN)*, pages 216–221, 2015.
- [12] Chen Ma and Yuhong Chi. Evaluation test and improvement of load balancing algorithms of nginx. *IEEE Access*, 10:14311–14324, 2022.

# Referências III

- [13] Ibrahim Mahmood, Siddeeq Ameen, Hajar Yasin, Naaman Omar, Shakir Kak, Zryan Najat, Azar Salih, Nareen O. M.Salim, and Dindar Ahmed. Web server performance improvement using dynamic load balancing techniques: A review. *Asian Journal of Research in Computer Science*, 06 2021.
- [14] Microsoft Corporation. Memory performance counters, April 2023. Referência oficial para os contadores de desempenho de memória no Windows.
- [15] Microsoft Corporation. Physicaldisk object, October 2023. Referência oficial para os contadores de desempenho do objeto PhysicalDisk, usado para monitoramento de I/O de disco.
- [16] R. Mirchandaney, D. Towsley, and J.A. Stankovic. Analysis of the effects of delays on load sharing. *IEEE Transactions on Computers*, 38(11), 1989.
- [17] NGINX. Http load balancer - nginx documentation, 2025. Accessed: 2025-03-27.

## Referências IV

- [18] A. Pasumpon Pandian, Xavier Fernando, and Wang Haoxiang, editors. *Computer Networks, Big Data and IoT: Proceedings of ICCBI 2021*, volume 117 of *Lecture Notes on Data Engineering and Communications Technologies*. Springer Nature Singapore, 2022.
- [19] Sam Preston. Finding the best servers to answer queries—edge dns and anycast, March 2021. Akamai Blog.
- [20] Iraj Sodagar. The mpeg-dash standard for multimedia streaming over the internet. *IEEE MultiMedia*, 18(4), 2011.
- [21] Thomas Stockhammer. Dynamic adaptive streaming over http: Standards and design principles. 02 2011.
- [22] The Linux Kernel Developers. I/o statistics fields for block devices, January 2011. Descreve os campos fornecidos pelo arquivo `/proc/diskstats`.
- [23] The Linux Kernel Developers. The `/proc` filesystem, March 2020. Documentação do kernel sobre a estrutura e os arquivos do sistema de arquivos `/proc`, incluindo `/proc/meminfo`.

# Referências V

- [24] Shiao-Li Tsao, Meng Chang Chen, Ming-Tat Ko, Jan-Ming Ho, and Yueh-Min Huang. Data allocation and dynamic load balancing for distributed video storage server. *Journal of Visual Communication and Image Representation*, 10(2):197–218, 1999.